@bybrayan.ai

GUÍA PRÁCTICA / EDICIÓN 2026

31

mejoras reales para acelerar tu app

De la interfaz al servidor: diagnostica, prioriza, optimiza y demuestra el resultado con métricas.

FRONTEND · RED · API · BASE DE DATOS · BACKEND · MEDICIÓN

UNA IDEA POR ENCIMA DE TODAS

Mide antes. Cambia con criterio. Vuelve a medir.

No necesitas aplicar las 31.

Necesitas identificar cuáles tienen impacto en tu aplicación, comprobarlas y cambiar solo lo que de verdad mejora.

CÓMO APROVECHAR ESTA GUÍA

Léela como una auditoría, no como una lista de tareas obligatorias. Cada recomendación incluye una comprobación y cuándo conviene no tocarla.

La ruta correcta en 3 fases

01 MEDIR	Establece una línea base reproducible: páginas críticas, dispositivo, red, métricas de laboratorio y datos reales si están disponibles.
02 DIAGNOSTICAR	Revisa los 31 puntos. Clasifica cada uno como ya resuelto, pendiente o no aplicable. Anexa una evidencia observable.
03 CAMBIAR	Prioriza por impacto y esfuerzo; pide aprobación, aplica un cambio por commit, ejecuta pruebas y compara el antes con el después.

Qué contiene este documento

- Checklist integral de **31 mejoras**, organizado por las seis capas del sistema.
- Recomendaciones específicas para **Next.js, Vercel, Supabase/Postgres y React**.
- Un **prompt maestro** y seis prompts de auditoría por capa para utilizar con un agente de código.
- Una **plantilla de informe**, lista para completar con mediciones y decisiones.

PRECAUCIÓN

No trabajes directamente sobre producción ni datos reales. Los cambios deben probarse en una rama o entorno de desarrollo, con una forma conocida de revertirlos.

Primero, define la línea base

Mide el comportamiento de las tres rutas más importantes y registra en qué condiciones las probaste.

Métricas de experiencia del usuario

MÉTRICA	UMBRAL BUENO	QUÉ OBSERVA
LCP	≤ 2,5 s	Cuándo aparece el contenido principal.
INP	≤ 200 ms	Qué tan rápido responde la interfaz a interacciones.
CLS	≤ 0,1	Cuánto se mueve inesperadamente la interfaz.

Los umbrales anteriores corresponden a Core Web Vitals y se evalúan idealmente al **percentil 75 (p75)** con datos de usuarios reales.

Separa laboratorio y uso real

- **Laboratorio:** Lighthouse móvil, DevTools y pruebas controladas. Permiten comparar versiones en condiciones similares.
- **Usuarios reales:** CrUX, PageSpeed Insights, Speed Insights o la librería `web-vitals`. Muestran la experiencia de visitas reales, cuando existen datos suficientes.
- **Servidor y datos:** trazas de peticiones, consultas lentas, carga inicial de JavaScript y latencias de servicios externos.

Ficha mínima de una medición

CAMPO	REGISTRA
Ruta analizada	Inicio / listado / flujo de conversión.
Entorno	Commit, rama, hosting y región.
Dispositivo	Modelo o perfil simulado; CPU y conexión.
Fuente	Laboratorio, usuarios reales o APM.
Resultado	Valor inicial, valor final y diferencia.

Nota: el objetivo de Lighthouse ≥ 90 puede servir como criterio interno, pero no sustituye a la evaluación de Core Web Vitals.

SECCIÓN 01 / 06

Frontend

Lo que el usuario ve e interactúa

10 MEJORAS · REVISIÓN POR EVIDENCIA

01 **Skeletons de carga** con la forma real del contenido

CÓMO COMPROBARLO DevTools → Network → "Slow 3G": el esqueleto aparece al instante y, al cargar, nada salta (CLS ≤ 0,1)

CUÁNDO NO APLICA Pantallas sin datos remotos

04 **Debounce** en búsqueda y autocompletado (≈ 250-400 ms)

CÓMO COMPROBARLO Escribe una palabra de 10 letras: debe salir 1 petición, no 10

CUÁNDO NO APLICA Búsquedas 100 % locales sin red

11 Dependencias sin usar eliminadas

CÓMO COMPROBARLO `npx depcheck` o `npx knip`; analizador de bundle (`@next/bundle-analyzer`, `vite-bundle-visualizer`)

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

14 **Interfaz optimista** con reversión si falla

CÓMO COMPROBARLO Al pulsar "me gusta", el contador cambia al instante; si el servidor falla, vuelve atrás y avisa

CUÁNDO NO APLICA Acciones críticas (pagos): ahí se espera confirmación

15 Lazy loading de imágenes fuera de pantalla

CÓMO COMPROBARLO `loading="lazy"` solo en imágenes bajo el pliegue. **Nunca** en la imagen principal (LCP): esa lleva `fetchpriority="high"`

CUÁNDO NO APLICA `next/image` es lazy por defecto; marca la principal con `priority`

21 Scripts de terceros después de la página

CÓMO COMPROBARLO Lighthouse: "Reduce the impact of third-party code". Carga chat/analítica/anuncios con `defer`, `async` o `next/script` con `strategy="lazyOnload"`

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

22 JavaScript dividido por página

CÓMO COMPROBARLO Analizador de bundle: cada ruta baja solo su código; los componentes pesados usan `import()` dinámico

CUÁNDO NO APLICA El App Router de Next.js ya divide por segmento de ruta; revisa solo los imports pesados

23 Listas largas virtualizadas

CÓMO COMPROBARLO Con 1.000 filas, el DOM solo tiene las visibles (TanStack Virtual, react-window)

CUÁNDO NO APLICA Listas cortas

26 Re-renders innecesarios eliminados

CÓMO COMPROBARLO React DevTools → Profiler: al cambiar un dato solo se renderiza lo que depende de él. Mide antes de poner `memo` en todo

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

29 Fuentes propias y pocos grosores

CÓMO COMPROBARLO No hay peticiones a servidores de fuentes externos; solo se cargan los grosores que se usan (p. ej. 400 y 700)

CUÁNDO NO APLICA `next/font` ya las descarga en el build y las sirve desde tu dominio

SECCIÓN 02 / 06

Red

Cómo se entregan los recursos

04 MEJORAS · REVISIÓN POR EVIDENCIA

02 **Imágenes en WebP o AVIF**, con `srcset`/`sizes` y `width`/`height`

CÓMO COMPROBARLO Lighthouse: “Serve images in modern formats” y “Properly size images”. Usa `<picture>` con respaldo: AVIF funciona desde Safari 16.1

CUÁNDO NO APLICA `next/image` ya sirve formatos modernos y tamaños responsive

06 **Estáticos desde una CDN**

CÓMO COMPROBARLO Cabeceras de respuesta de un CSS/JS: hay caché de borde (p. ej. `x-vercel-cache: HIT`, `cf-cache-status: HIT`)

CUÁNDO NO APLICA Vercel, Netlify y Cloudflare Pages ya lo hacen

09 **Compresión Brotli o Gzip**

CÓMO COMPROBARLO Cabecera `Content-Encoding: br` o `gzip` en HTML, JS, CSS y JSON

CUÁNDO NO APLICA Vercel la aplica sola (prefiere Brotli si el navegador lo admite)

27 **Caché del navegador para estáticos**

CÓMO COMPROBARLO Archivos con hash en el nombre (`app.3f9a.js`) servidos con `Cache-Control: public, max-age=31536000, immutable`

CUÁNDO NO APLICA Los frameworks modernos ya ponen el hash; revisa las imágenes y fuentes propias

SECCIÓN 03 / 06

API

Cómo fluye la información entre servicios

05 MEJORAS · REVISIÓN POR EVIDENCIA

03 **Sin llamadas duplicadas** a la API

CÓMO COMPROBARLO Network: la misma URL no aparece varias veces en una misma carga. Comparte la petición (caché de TanStack Query/SWR o un único fetch en el padre)

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

05 **Paginación** (o scroll infinito por páginas)

CÓMO COMPROBARLO Ninguna respuesta trae miles de filas; la API acepta `limit` y `cursor`/`page`

CUÁNDO NO APLICA Listas cortas y acotadas

12 **Timeout en APIs externas**, con plan B

CÓMO COMPROBARLO Simula una API lenta: a los N segundos se corta y la página muestra "no disponible · reintentar" en vez de colgarse (`AbortSignal.timeout(ms)` en fetch)

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

13 **Streaming de respuestas de IA** token a token

CÓMO COMPROBARLO La primera palabra aparece en < 1 s aunque la respuesta completa tarde más

CUÁNDO NO APLICA Respuestas cortas que llegan de una

16 **Peticiones independientes en paralelo**

CÓMO COMPROBARLO En la cascada de Network no hay escaleras de peticiones que no dependen entre sí (`Promise.all`)

CUÁNDO NO APLICA Peticiones que dependen del resultado de otra

SECCIÓN 04 / 06

Base De Datos

Consultas y conexiones sin desperdicio

04 MEJORAS · REVISIÓN POR EVIDENCIA

08 Consultas N+1 arregladas

CÓMO COMPROBARLO Log de consultas: una lista de N elementos no lanza N+1 consultas. Usa join, `IN (…)` o el `include`/`select` anidado de tu ORM

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

17 Índices para las consultas comunes

CÓMO COMPROBARLO `EXPLAIN ANALYZE` de la consulta: usa `Index Scan` y no `Seq Scan` en tablas grandes

CUÁNDO NO APLICA Tablas pequeñas (pocos miles de filas)

24 Solo las columnas necesarias

CÓMO COMPROBARLO Ninguna consulta de lista usa `SELECT *`; los campos privados no salen de la base de datos

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

25 Pool de conexiones

CÓMO COMPROBARLO En serverless, la conexión pasa por un pooler. En Supabase: Supavisor en modo transacción, **puerto 6543** (no admite sentencias preparadas: en Prisma, `pgbouncer=true` y `connection_limit=1` para empezar)

CUÁNDO NO APLICA Servidores con una sola conexión persistente bien configurada

SECCIÓN 05 / 06

Backend

Trabajo del servidor y capacidad

05 MEJORAS · REVISIÓN POR EVIDENCIA

07 **Cola de trabajos** para tareas lentas (PDF, correos, imágenes, IA larga)

CÓMO COMPROBARLO Ninguna petición HTTP tarda más de ~1-2 s por trabajo pesado; el usuario recibe "en cola" y el resultado llega después

CUÁNDO NO APLICA Apps sin tareas pesadas

10 **Caché (Redis) para datos muy consultados**, siempre con **TTL** e invalidación

CÓMO COMPROBARLO Segunda petición igual: no toca la base de datos. Comprueba que al cambiar el dato la caché se invalida o caduca

CUÁNDO NO APLICA Datos que cambian en cada petición

19 **Servidor y base de datos en la misma región**

CÓMO COMPROBARLO Región de las funciones (p. ej. Vercel) = región del proyecto de base de datos

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

20 **Render en el servidor o en el build** (SSR/SSG)

CÓMO COMPROBARLO Desactiva JavaScript: la página igual muestra su contenido principal

CUÁNDO NO APLICA Paneles privados muy interactivos

28 **Tráfico repartido entre varias instancias**

CÓMO COMPROBARLO Con un VPS propio: balanceador + 2 o más instancias con *health check*

CUÁNDO NO APLICA En serverless (Vercel, Netlify) la plataforma ya escala y reparte

SECCIÓN 06 / 06

Medición

Pruebas que demuestran la mejora

03 MEJORAS · REVISIÓN POR EVIDENCIA

18 **Trazas de peticiones lentas** (OpenTelemetry)

CÓMO COMPROBARLO Cada petición lenta muestra sus tramos (auth, consulta, API externa, render) y se ve cuál se lleva el tiempo

CUÁNDO NO APLICA Si tu hosting o APM ya trae trazas, úsalo

30 **Velocidad de usuarios reales**

CÓMO COMPROBARLO Core Web Vitals al percentil 75: **LCP ≤ 2,5 s · INP ≤ 200 ms · CLS ≤ 0,1** (PageSpeed Insights/CrUX, Vercel Speed Insights o la librería `web-vitals`)

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

31 **Pruebas en celular de gama baja y red lenta**

CÓMO COMPROBARLO DevTools: CPU 4× más lenta + red 3G; o un Android barato real. La app sigue siendo usable

CUÁNDO NO APLICA Sin excepción general: evalúa según tu contexto y evidencia.

Cómo priorizar sin sobreoptimizar

No existe una misma lista de prioridades para todos los proyectos.

REGLA DE DECISIÓN

Primero identifica el cuello de botella medido. Después estima impacto, esfuerzo, riesgo y dependencia técnica. No implementes algo solo porque aparece en el checklist.

Orden práctico de trabajo

1 · Detecta	Identifica la página, interacción o endpoint donde el usuario realmente percibe el problema.
2 · Acota	Encuentra si el costo está en JavaScript, imágenes, red, consultas, APIs externas o render.
3 · Estima	Da prioridad a alto impacto y bajo esfuerzo, con riesgo bajo y reversión sencilla.
4 · Ejecuta	Cambia una cosa a la vez; verifica tests y captura métricas comparables.
5 · Decide	Conserva el cambio si el beneficio medido justifica su complejidad y costo.

No hagas esto por reflejo

- Agregar Redis a una aplicación cuya base de datos responde bien y no presenta lecturas repetidas.
- Introducir colas, microservicios o balanceadores donde no hay carga que lo justifique.
- Poner `memo` en cada componente o índices indiscriminados sin perfiles y planes de consulta.
- Activar lazy loading en la imagen principal o usar actualizaciones optimistas en pagos sin confirmación.

Tu stack ya resuelve parte del trabajo

Comprueba las capacidades de tus herramientas antes de reinventar optimizaciones.

NEXT.JS · APP ROUTER

22 · Code splitting Segmentos de ruta divididos automáticamente; revisar imports pesados dentro de cada página.

29 · Fuentes `next/font` sirve fuentes desde el dominio; seleccionar solo pesos necesarios.

02 / 15 · Imágenes `next/image` optimiza tamaños y formatos; priorizar la imagen LCP en lugar de cargarla de forma diferida.

20 · Render Server Components reducen JS del cliente y algunas páginas pueden prerenderizarse.

VERCEL

09 · Compresión La CDN puede entregar Brotli o gzip; comprobar `Content-Encoding`.

06 / 28 · Distribución Los estáticos usan CDN y el proveedor gestiona el escalado según el tipo de despliegue.

19 · Región Alinear ubicación de funciones y base de datos cuando sea posible.

SUPABASE / POSTGRES

25 · Pooler En entornos serverless revisar Supavisor en modo transacción, puerto 6543, y compatibilidad con el ORM.

17 · Índices Validar consultas importantes con `EXPLAIN ANALYZE`.

24 · Selección Pedir campos específicos, no `SELECT *` en listados.

REACT

26 · Re-renders Usar React DevTools Profiler antes de decidir sobre `memo`/`useMemo`/`useCallback`.

23 · Virtualización TanStack Virtual o react-window cuando las listas son muy largas.

03 / 14 · Datos TanStack Query o SWR para compartir consultas, y manejar actualizaciones optimistas si procede.

Las capacidades dependen de versiones y configuración. Consulta la documentación oficial de tu versión antes de ajustar conexiones, caché o opciones de imágenes.

Prompt maestro para tu agente de código

Pégalo desde la raíz de tu proyecto. Obliga a diagnosticar antes de modificar.

MODO DE USO

Este prompt está pensado para Claude Code o un agente similar. Exige evidencia, aprobación explícita y cambios reversibles. Dale acceso al código de desarrollo, nunca a secretos de producción.

```
Quiero mejorar el rendimiento de esta aplicación con evidencia, no con suposiciones.

REGLAS
- Trabaja en una rama nueva. No toques producción, secretos ni datos reales.
- No elimines archivos o datos sin autorización.
- No modifiques código hasta que apruebe tu diagnóstico.
- Si el framework o hosting ya resuelve algo, verifica y no lo dupliques.
- Un cambio por commit; ejecuta tests, mide y revierte si falla.
- Si una métrica no está disponible, decláralo; no inventes resultados.

FASE 1 / MEDIR (SOLO LECTURA)
1. Detecta framework, hosting, base de datos, librerías y versiones.
2. Analiza las 3 rutas más importantes con Lighthouse móvil, si está disponible.
3. Recoge LCP, INP y CLS de usuarios reales al p75 si existe esa información.
4. Registra tamaño inicial de JS y consultas más lentas/repetidas si hay acceso.
5. Guarda la línea base en PERF_BASELINE.md con entorno y metodología.

FASE 2 / DIAGNOSTICAR
Audita estos 31 puntos de la guía:
01 Skeletons; 02 imágenes modernas; 03 API sin duplicados;
04 debounce; 05 paginación; 06 CDN; 07 colas;
08 N+1; 09 compresión; 10 caché/TTL; 11 dependencias;
12 timeouts; 13 streaming IA; 14 interfaz optimista;
15 lazy loading; 16 paralelismo; 17 índices; 18 trazas;
19 misma región; 20 SSR/SSG; 21 scripts terceros;
22 code splitting; 23 virtualización; 24 columnas necesarias;
25 pool de conexiones; 26 re-renders; 27 caché navegador;
28 balanceo; 29 fuentes locales; 30 métricas reales;
31 móvil de gama baja y red lenta.

Para cada punto: estado (ya está / falta / no aplica), evidencia
(archivo:línea, configuración o medición), impacto, esfuerzo y riesgo.
Ordena propuestas por impacto/ esfuerzo y muestra el diagnóstico.
ESPERA mi aprobación para empezar cambios.

FASE 3 / IMPLEMENTAR SOLO CON APROBACIÓN
- Aplica una mejora por commit, documentando motivo y cambios.
- Ejecuta tests y vuelve a medir la dimensión afectada.
- Si empeora o rompe funcionalidad, revierte el cambio.
- Rellena el informe final con antes, después y descartes.
- No afirmes mejoras sin datos comparables.
```

Prompts específicos por capa

Si tu aplicación es grande, trabaja una capa por sesión en lugar de auditarlo todo al mismo tiempo.

Encabezado común para todos

```
Trabaja en una rama nueva. No toques producción ni datos reales. Primero mide y muestra el
diagnóstico con evidencia (archivo:línea o medición). No cambies nada hasta que diga "adelante". Un
cambio por commit; si se rompe, revierte.
```

01 / FRONTEND

```
Revisa skeletons y CLS, debounce, dependencias sin uso, interfaz optimista con reversión, lazy
loading solo bajo el pliegue, scripts de terceros, imports dinámicos, virtualización, re-renders
medidos con Profiler y fuentes locales. Indica evidencia, riesgos, impacto y esfuerzo.
```

02 / RED

```
Revisa AVIF/WebP, srcset/sizes y dimensiones; CDN para estáticos; compresión Brotli/gzip; caché
immutable para archivos con hash. Distingue lo que el hosting hace automáticamente de lo que falta.
```

03 / API

```
Busca peticiones duplicadas, endpoints sin paginación, APIs externas sin timeout ni reintento seguro,
respuestas de IA sin streaming y peticiones independientes ejecutadas secuencialmente. Propón cambios
con evidencia.
```

04 / BASE DE DATOS

```
En desarrollo inspecciona consultas N+1, planes EXPLAIN ANALYZE para índices útiles, SELECT * en
listados y pool de conexiones cuando se use serverless. Revisa compatibilidad del ORM y nunca cambies
producción.
```

05 / BACKEND

```
Identifica tareas pesadas dentro de HTTP, caché que realmente pueda reducir lecturas, desajuste de
regiones, páginas aptas para SSR/SSG y necesidad real de balanceo. No agregues infraestructura si la
plataforma ya resuelve el problema.
```

06 / MEDICIÓN

```
Documenta Core Web Vitals de usuarios reales al p75, trazas de endpoints lentos y prueba de las tres
rutas importantes con CPU 4× lenta y red 3G. Separa laboratorio de datos reales y completa el informe
final.
```

Plantilla de informe de rendimiento

Duplica esta sección en tu propio documento o complétala en Word para registrar decisiones y resultados.

Ficha de auditoría

App: _____ Fecha: _____

Rama / commit: _____ Responsable: _____

Dispositivo y red: _____ Fuente de datos: _____

01 / Línea base y resultado

MÉTRICA	REFERENCIA	ANTES	DESPUÉS
LCP · p75	≤ 2,5 s	_____	_____
INP · p75	≤ 200 ms	_____	_____
CLS · p75	≤ 0,1	_____	_____
Lighthouse móvil	Meta interna ≥ 90	_____	_____
JS inicial	Menor es mejor*	_____	_____
Consulta más lenta	Según línea base	_____	_____

*El tamaño de JS y el tiempo de consultas se evalúan según cada aplicación; no existe un valor universal en esta guía.

02 / Mejoras aplicadas

#	CAMBIO / COMMIT	ANTES	DESPUÉS
___	_____	_____	_____
___	_____	_____	_____
___	_____	_____	_____
___	_____	_____	_____

Informe / decisiones y pruebas

Una mejora descartada con evidencia también es una buena decisión.

03 / Revisado y descartado

#	PUNTO	MOTIVO / EVIDENCIA
___	_____	Ya lo hace el stack / no aplica / sin beneficio
___	_____	_____
___	_____	_____

04 / Próximos pendientes

#	MEJORA	IMPACTO	ESFUERZO	RESPONSABLE
___	_____	A / M / B	A / M / B	_____
___	_____	A / M / B	A / M / B	_____
___	_____	A / M / B	A / M / B	_____

05 / Validación bajo condiciones difíciles

CPU 4× más lenta	[ ] Usable [ ] Con problemas Observaciones: _____
Red 3G lenta	[ ] Usable [ ] Con problemas Observaciones: _____
Pruebas	[ ] Correctas [ ] Fallidas [ ] No ejecutadas
Decisión final	[ ] Conservar [ ] Revertir [ ] Repetir medición

CRITERIO DE CIERRE

La optimización queda documentada solo cuando puedes mostrar qué se cambió, por qué se cambió, cómo se probó y qué ocurrió con la métrica relevante.

Documentación de referencia

Recursos oficiales mencionados en los materiales para contrastar versiones, compatibilidad y métodos de medición.

Core Web Vitals — umbrales
https://web.dev/articles/defining-core-web-vitals-thresholds

Core Web Vitals — fundamentos
https://web.dev/articles/vitals

Interacción — INP
https://web.dev/articles/inp

LCP y lazy loading
https://web.dev/articles/lcp-lazy-loading

MDN — formatos de imagen
https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Image_types

Next.js — Font Optimization
https://nextjs.org/docs/app/getting-started/fonts

Next.js — navegación y rutas
https://nextjs.org/learn/dashboard-app/navigating-between-pages

Vercel — compresión CDN
https://vercel.com/docs/how-vercel-cdn-works/compression

Supabase — conexión a Postgres
https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase — Prisma troubleshooting
https://supabase.com/docs/guides/database/prisma/prisma-troubleshooting

Antes de dar por terminada la auditoría

- ¿Se registró la línea base con condiciones reproducibles?
- ¿Cada punto auditado tiene estado y evidencia?
- ¿El cambio aprobado se probó fuera de producción?
- ¿Se puede mostrar el efecto del cambio sin inventar números?

EL MEJOR RENDIMIENTO NO ES UNA CASUALIDAD.

Es el resultado de medir, entender el cuello de botella y optimizar lo que importa.

¿TE FUE ÚTIL ESTA GUÍA?

Sígueme en Instagram para más contenido sobre desarrollo de software, IA aplicada y optimización de aplicaciones.

@bybrayan.ai

instagram.com/bybrayan.ai

Guárdala. Aplícala. Mide la diferencia.